WINDOW FUNCTION

A **window function** in SQL is a function that performs calculations across a defined set of rows (called a *window*) while still returning **each individual row in the result set**.

Unlike GROUP BY, which reduces multiple rows into a single summary row, window functions **preserve row-level detail and add analytical computations on top of it**.

They are commonly used for:

- Rankings
- running totals
- partition-based averages
- comparative analytics
- KPI calculations

SHORT NOTES ON EACH SCRIPT

SECTION 1: INTRODUCTION

```
SELECT *
FROM employees;
```

What it does:

- Displays all employee records
- Acts as the base dataset for demonstrating window functions
- No transformation is applied

SECTION 2: GROUP BY vs WINDOW FUNCTION

GROUP BY version

```
SELECT d.department_name,
      ROUND(AVG(jr.base_salary), 2) AS avg_salary
...
GROUP BY d.department_name;
```

What it does:

- Groups employees by department
- Calculates average salary per department
- Returns **one row per department**
- Original employee-level detail is lost

WINDOW FUNCTION version

```
AVG(jr.base_salary) OVER()
```

What it does:

- Calculates **overall company average salary**
- Keeps all employee rows
- Adds the same computed value to every row

Key idea:

Aggregation without losing detail rows

SECTION 3: PARTITION BY (GROUP-LEVEL ANALYTICS)

```
AVG(jr.base_salary) OVER(PARTITION BY d.department_name)
```

What it does:

- Splits data by department
- Computes average salary per department
- Keeps each employee row intact

Result:

- Each employee sees their department's average salary

SECTION 4: RUNNING TOTAL (CUMULATIVE SUM)

```
SUM(jr.base_salary)

OVER(PARTITION BY d.department_name ORDER BY e.employee_id)
```

What it does:

- Calculates cumulative salary within each department
- Adds salaries progressively based on employee order

Key idea:

- Tracks how totals build over time or sequence

SECTION 5: ROW_NUMBER

```
ROW_NUMBER() OVER(
    PARTITION BY d.department_name
    ORDER BY jr.base_salary DESC
)
```

What it does:

- Assigns a unique sequence number within each department
- Highest salary gets rank 1
- No duplicates allowed

Key idea:

Every row gets a unique position

SECTION 6: RANK VS DENSE_RANK COMPARISON

ROW_NUMBER

- Unique numbering
- No duplicates
- Always sequential

RANK

```
RANK() OVER(...)
```

What it does:

- Assigns ranking based on salary
- Ties receive same rank
- Skips numbers after ties

DENSE_RANK

```
DENSE_RANK() OVER(...)
```

What it does:

- Same as RANK
- BUT no gaps in ranking numbers

Summary Example:

Salary	ROW_NUMBER	RANK	DENSE_RANK
140K	1	1	1
120K	2	2	2
120K	3	2	2
100K	4	4	3

KEY CONCEPT SUMMARY

WINDOW FUNCTIONS

- Perform calculations without collapsing rows

OVER()

- Defines the scope of calculation

PARTITION BY

- Splits data into groups (like departments)

ORDER BY (inside OVER)

- Defines ranking or sequence logic

ROW_NUMBER

- Unique numbering per group

RANK

- Ranking with gaps

DENSE_RANK

- Ranking without gaps

BUSINESS USE CASES (TECHSPHERE CONTEXT)

Window functions are widely used for:

- employee performance ranking
- salary benchmarking per department
- payroll analytics
- KPI dashboards
- HR reporting systems
- executive decision support systems

FINAL INSIGHT

Window functions enable advanced analytics by combining aggregation logic with row-level detail, making them essential for modern SQL-based reporting and business intelligence systems.